

Deep learning

Master's Degree in Artificial Intelligence

2025/26

Unit 5D – Diffusion models

Index

5D.1 Introduction

5D.2 The theory behind diffusion models

5D.3 Examples of diffusion models

5D.4 Stable Diffusion

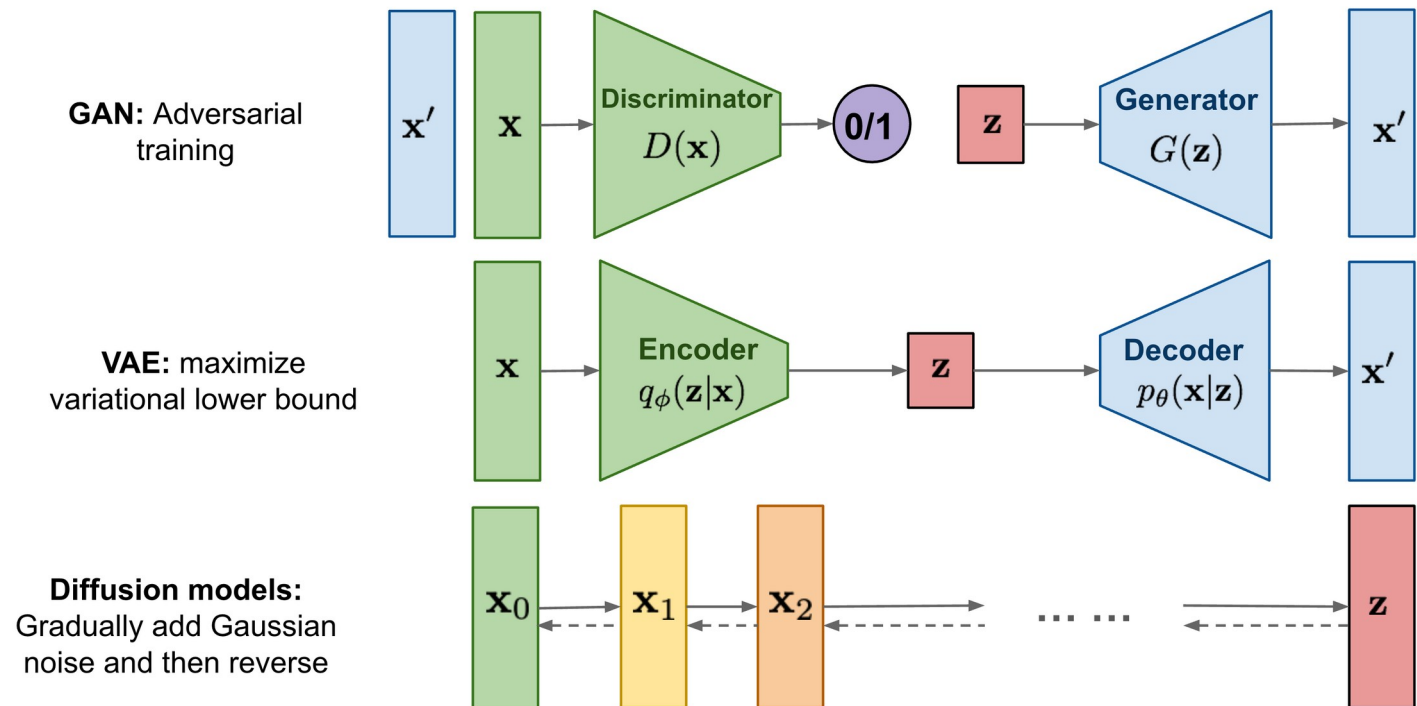
5D.5 Stable Diffusion at work

5D.1 Introduction

Diffusion models are generative models.

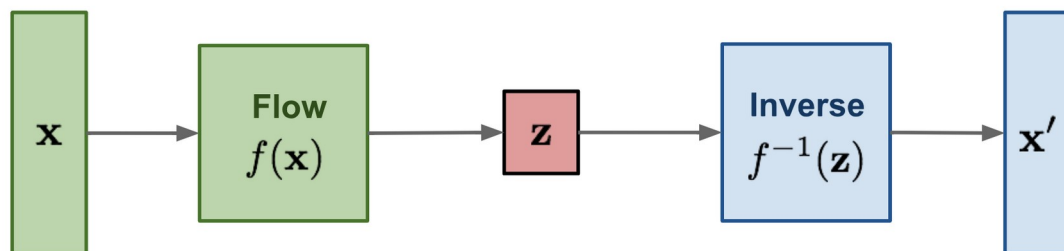
The main four categories of generative neural networks are:

- Variational Autoencoders (VAE)
- Generative Adversarial Networks (GANs)
- Diffusion models
- *Flow-based models*



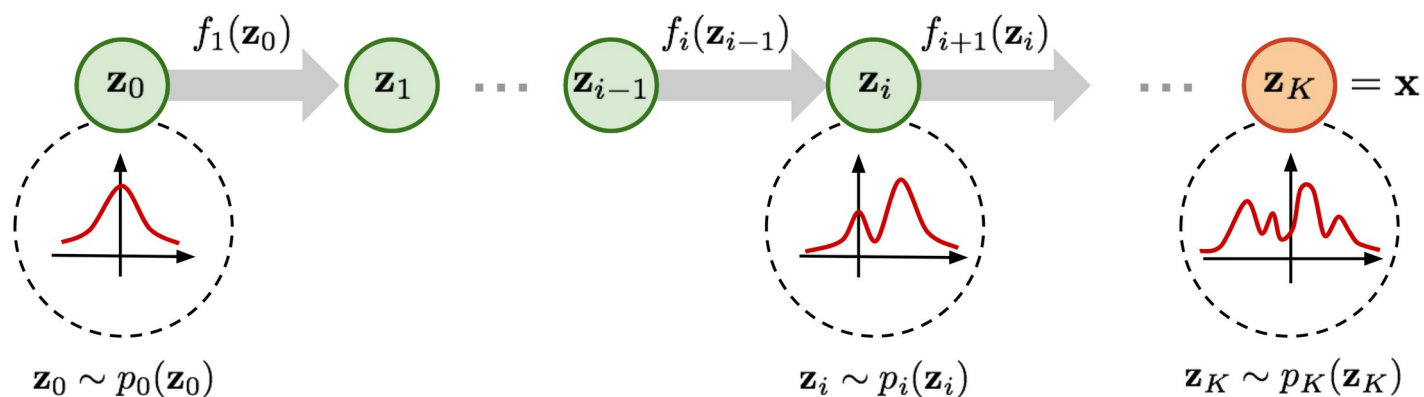
Flow-based models

**Flow-based
generative models:**
minimize the negative
log-likelihood

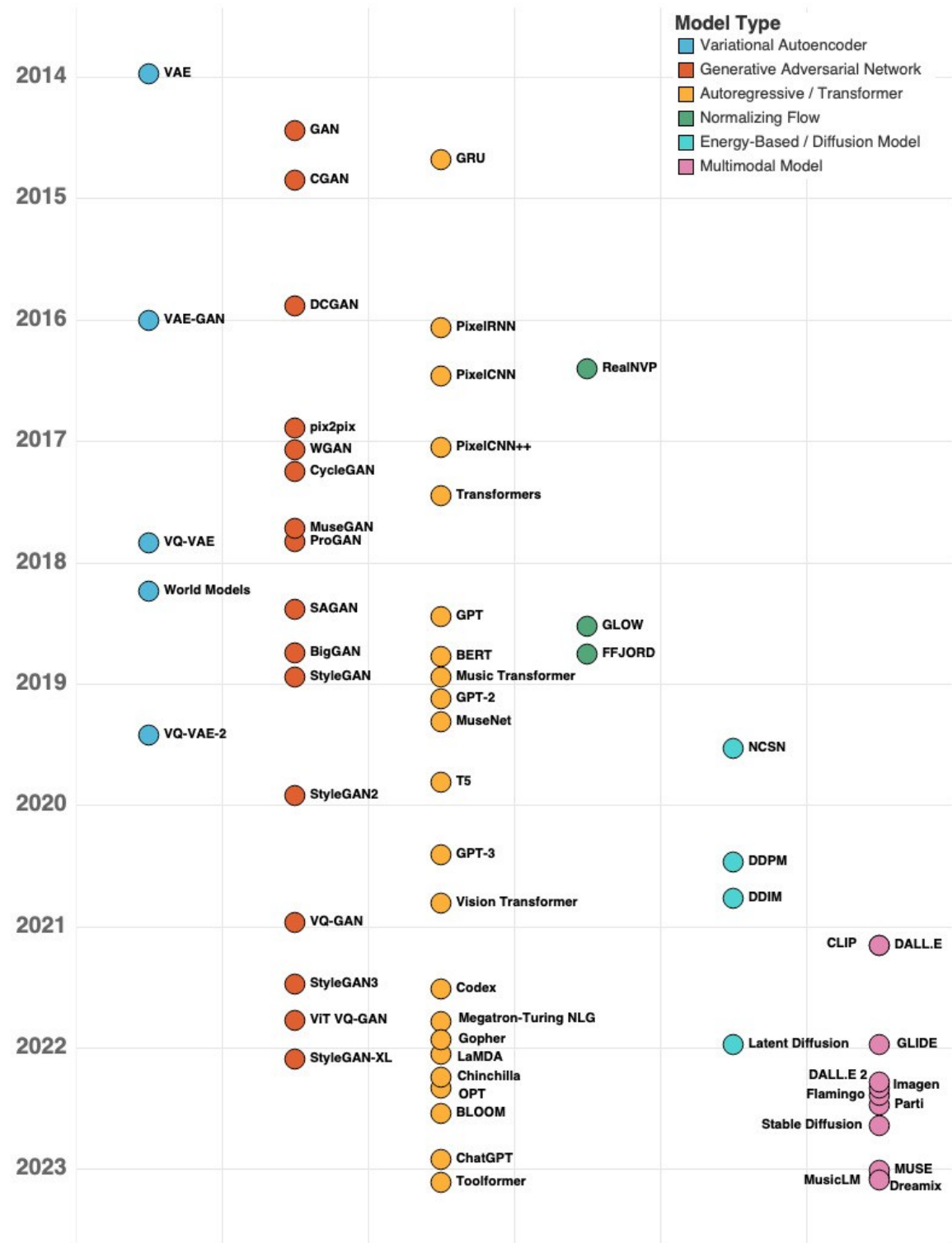


A flow-based generative model is constructed by a sequence of **invertible transformations**. Unlike VAEs or GANs, **the model explicitly learns the data distribution**:

- Starting point: a simple distribution (usually a normal one)
- Apply a series of invertible transformations to increase the complexity of the distribution

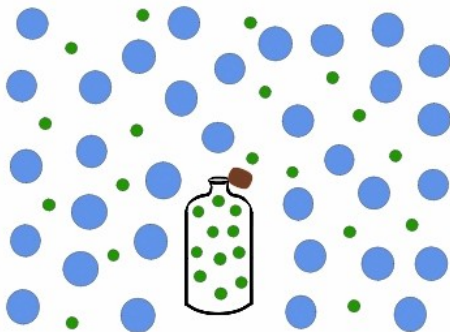


A history of generative neural networks



In machine learning, **diffusion models**, also known as diffusion probabilistic models, are a class of latent variable models. They are **Markov chains** trained using variational inference. The goal of diffusion models is to learn the latent structure of a dataset by modeling the way in which data points diffuse through the latent space. In computer vision, this means that a neural network is trained to denoise images blurred with Gaussian noise by learning to reverse the diffusion process.

Diffusion models were introduced in 2015 with a motivation from non-equilibrium thermodynamics.



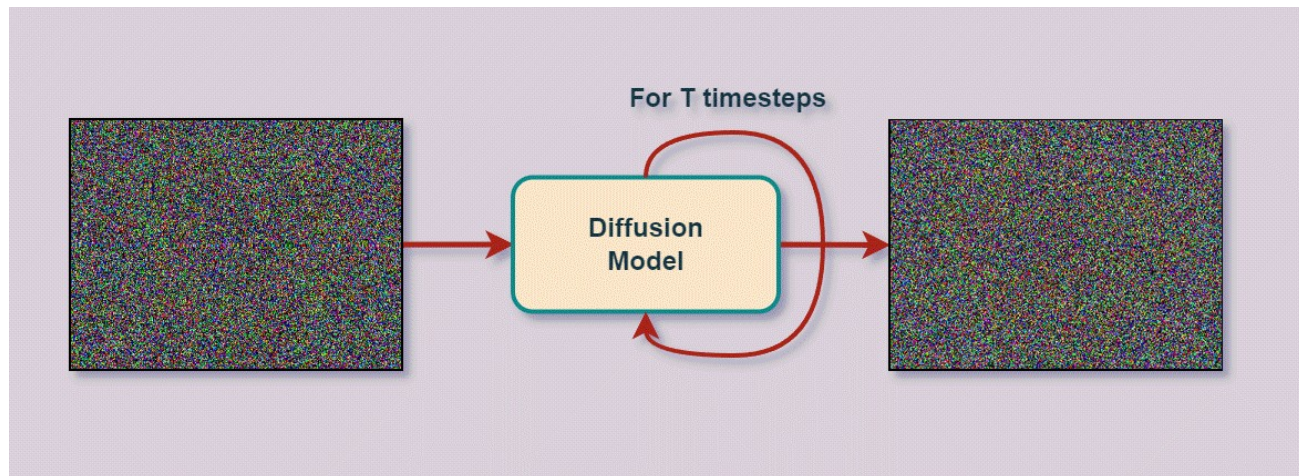
The diffusion method can be summarized as follows:

...systematically and slowly destroy the structure in a data distribution through an iterative forward diffusion process. We then learn a reverse diffusion process that restores structure in data, yielding a highly flexible and tractable generative model of the data. This approach allows us to rapidly learn, sample from, and evaluate probabilities in deep generative models ...

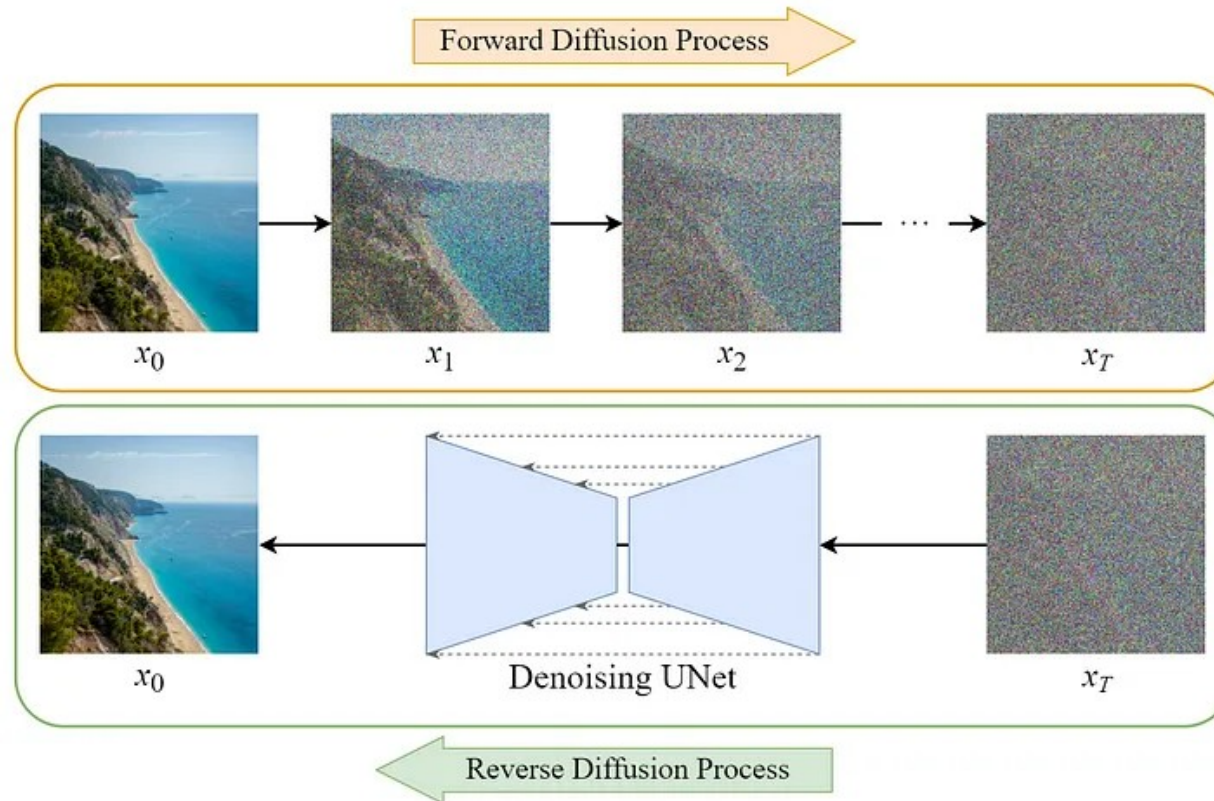
– Deep Unsupervised Learning using Nonequilibrium Thermodynamics, 2015

Diffusion models can generate images conditionally or unconditionally

- **Unconditional image generation**: the model converts noise into any “random representative data sample.” The generation process is not controlled or guided, and the model can generate an image of any nature.
- **Conditional image generation**: the model is provided additional information via text or class labels (like in conditional GANs).



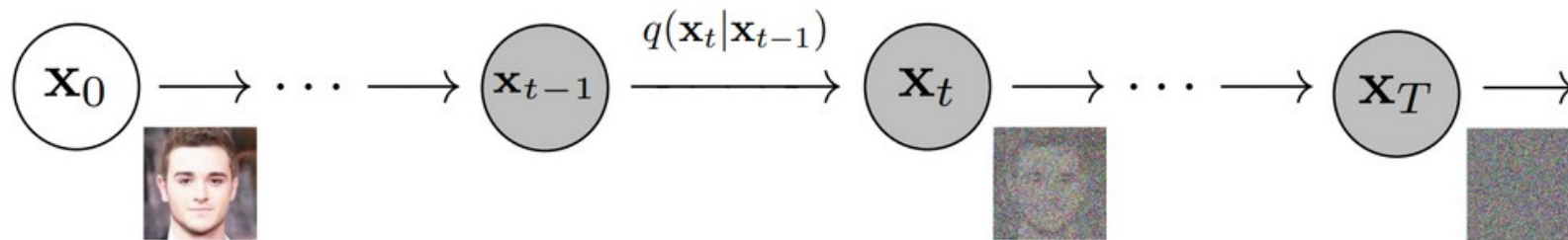
Diffusion models try to build a model that learns the decay of information due to noise. Then, it is possible to reverse the process and recover the information back from the noise



- **Forward Diffusion Process:** gaussian noise is introduced successively until the data becomes all noise.
- **Reverse Diffusion Process:** this process learns how to undo using a neural network model. In theory, applying this model could obtain an image that looks as if it were drawn from the training set

5D.2 The theory behind diffusion models

The forward diffusion process



The goal is to corrupt an image over a large number of steps T

Each step adds a little of gaussian noise producing noisier and noisier samples ($x_1, x_2, \dots x_T$)

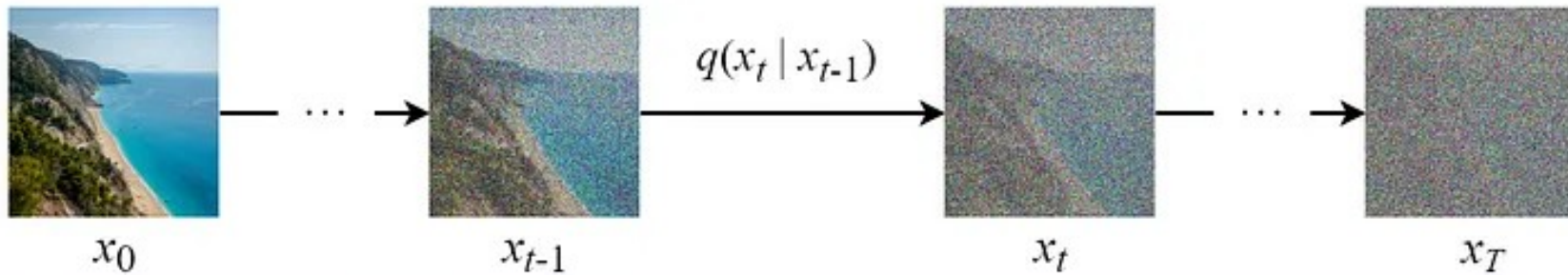
At the end we have gaussian noise

$$x_t = \sqrt{1 - \beta_t} x_{t-1} + \sqrt{\beta_t} \epsilon_{t-1}$$

where ϵ_{t-1} is a standard Gaussian with zero mean and unit variance and β_t is the variance of the added noise

The function q defines each step:

$$q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t \mathbf{I})$$



Distribution of the
noised images

Output

Mean μ_t

Variance Σ_t

$$q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t I)$$

Notations:

t : time step (from 0 to T)

x_0 : a data sampled from the real data distribution $q(x)$ (i.e. $x_0 \sim q(x)$)

β_t : variance schedule ($0 \leq \beta_t \leq 1$, and $\beta_0 = \text{small number}$, $\beta_T = \text{large number}$)

I : identity matrix

- β_t is the variance of the noise we add to the image x_{t-1}
- x_{t-1} is scaled so the variances of x_{t-1} and x_t are the same
- x_{t-1} follows a normal gaussian, then x_t follows a normal gaussian too

The forward diffusion process is fixed

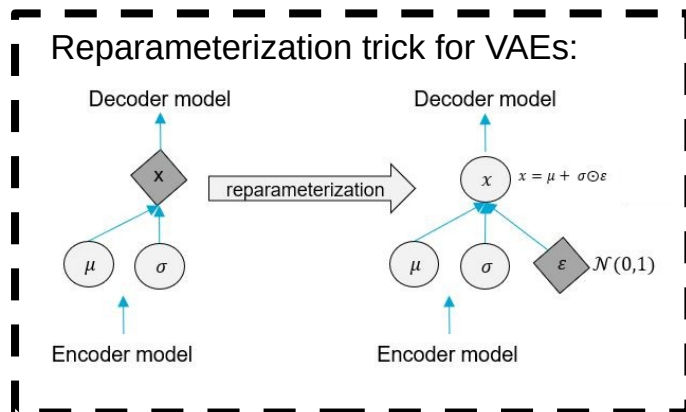
The reparameterization trick

This trick allows to obtain any x_t from x_0 , without all the intermediate steps

$$\alpha_t = 1 - \beta_t$$

$$\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$$

$$q(x_t | x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) \mathbf{I})$$



$$\begin{aligned}
 x_t &= \sqrt{1 - \beta_t} x_{t-1} + \sqrt{\beta_t} \varepsilon_{t-1} & \varepsilon_0, \dots, \varepsilon_{t-2}, \varepsilon_{t-1} &\sim \mathcal{N}(0, I) \\
 &= \sqrt{\alpha_t} x_{t-1} + \sqrt{1 - \alpha_t} \varepsilon_{t-1} & \bar{\varepsilon}_0, \dots, \bar{\varepsilon}_{t-2}, \bar{\varepsilon}_{t-1} &\sim \mathcal{N}(0, I) \\
 &= \sqrt{\alpha_t} \left(\sqrt{\alpha_{t-1}} x_{t-2} + \sqrt{1 - \alpha_{t-1}} \varepsilon_{t-2} \right) + \sqrt{1 - \alpha_t} \varepsilon_{t-1} & \varepsilon &\sim \mathcal{N}(0, I) \\
 &= \sqrt{\alpha_t \alpha_{t-1}} x_{t-2} + \sqrt{\alpha_t (1 - \alpha_{t-1})} \varepsilon_{t-2} + \sqrt{1 - \alpha_t} \varepsilon_{t-1} & \alpha_t &= 1 - \beta_t \\
 & & \bar{\alpha}_t &= \prod_{i=1}^t \alpha_i
 \end{aligned}$$

Reparameterization Trick

Underlying Normal Distribution

$$0 + \sqrt{\alpha_t (1 - \alpha_{t-1})} \varepsilon_{t-2} \longrightarrow X \sim \mathcal{N}(0, \alpha_t (1 - \alpha_{t-1}) I)$$

$$0 + \sqrt{1 - \alpha_t} \varepsilon_{t-1} \longrightarrow Y \sim \mathcal{N}(0, (1 - \alpha_t) I)$$

Recall:

$$\text{If } X \sim \mathcal{N}(\mu_X, \sigma_X^2) \quad Y \sim \mathcal{N}(\mu_Y, \sigma_Y^2)$$

$$Z = X + Y$$

$$\text{Then } Z \sim \mathcal{N}(\mu_X + \mu_Y, \sigma_X^2 + \sigma_Y^2)$$

$$\begin{aligned}
 Z &\sim \mathcal{N}(0, \sigma_X^2 + \sigma_Y^2) & \sigma_X^2 + \sigma_Y^2 &= \alpha_t (1 - \alpha_{t-1}) + (1 - \alpha_t) \\
 Z &\sim \mathcal{N}(0, (1 - \alpha_t \alpha_{t-1}) I) & &= \cancel{\alpha_t} - \alpha_t \alpha_{t-1} + 1 - \cancel{\alpha_t} \\
 & & &= 1 - \alpha_t \alpha_{t-1}
 \end{aligned}$$

Reparameterization Trick

$$0 + \sqrt{1 - \alpha_t \alpha_{t-1}} \bar{\varepsilon}_{t-2}$$

$$= \sqrt{\alpha_t \alpha_{t-1}} x_{t-2} + \sqrt{1 - \alpha_t \alpha_{t-1}} \bar{\varepsilon}_{t-2}$$

$\vdots$

$$= \sqrt{\alpha_t \alpha_{t-1} \dots \alpha_1} x_0 + \sqrt{1 - \alpha_t \alpha_{t-1} \dots \alpha_1} \varepsilon$$

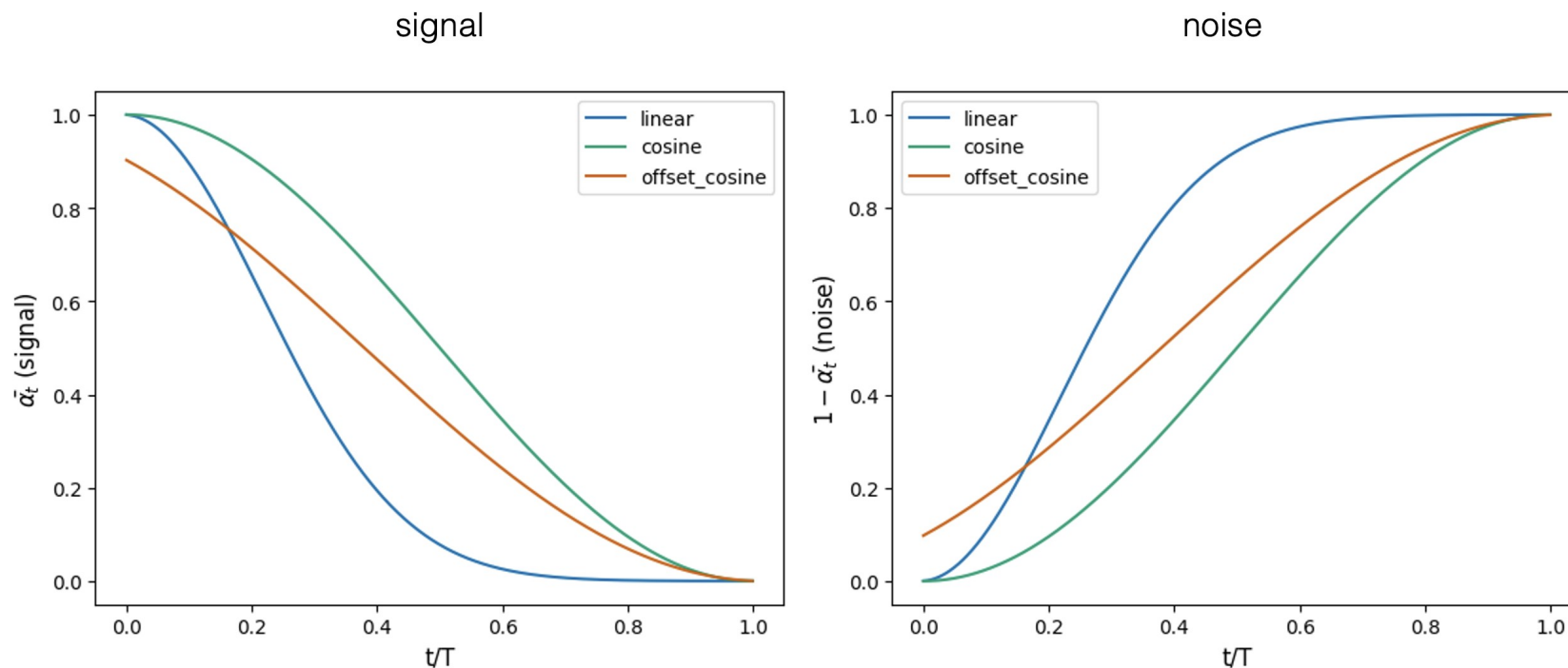
$$= \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon$$

Diffusion schedules

The sequence $(\beta_1, \beta_2, \dots, \beta_T)$ create a diffusion schedule

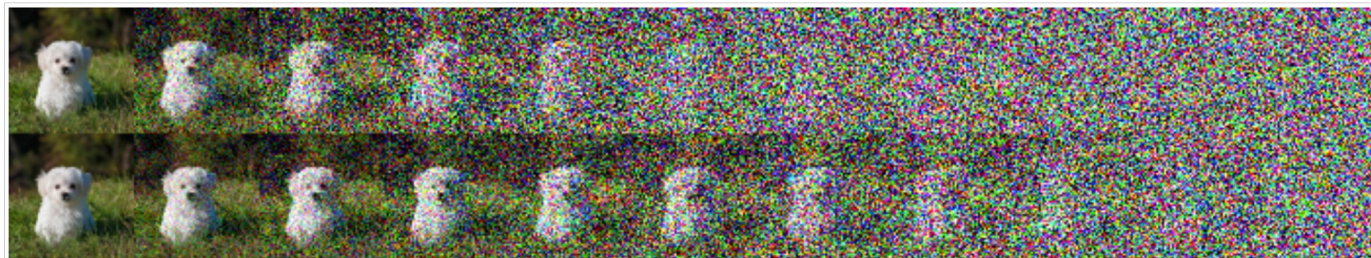
- ▶ The initial proposal is a linear schedule $\beta_1=0.0001 \dots \beta_T=0.02$
- ▶ Later proposals: cosine diffusion schedule with some offset to prevent very small steps at the beginning

$$\bar{\alpha}_t = \cos^2\left(\frac{t}{T} \times \frac{\pi}{2}\right)$$



linear

cosine



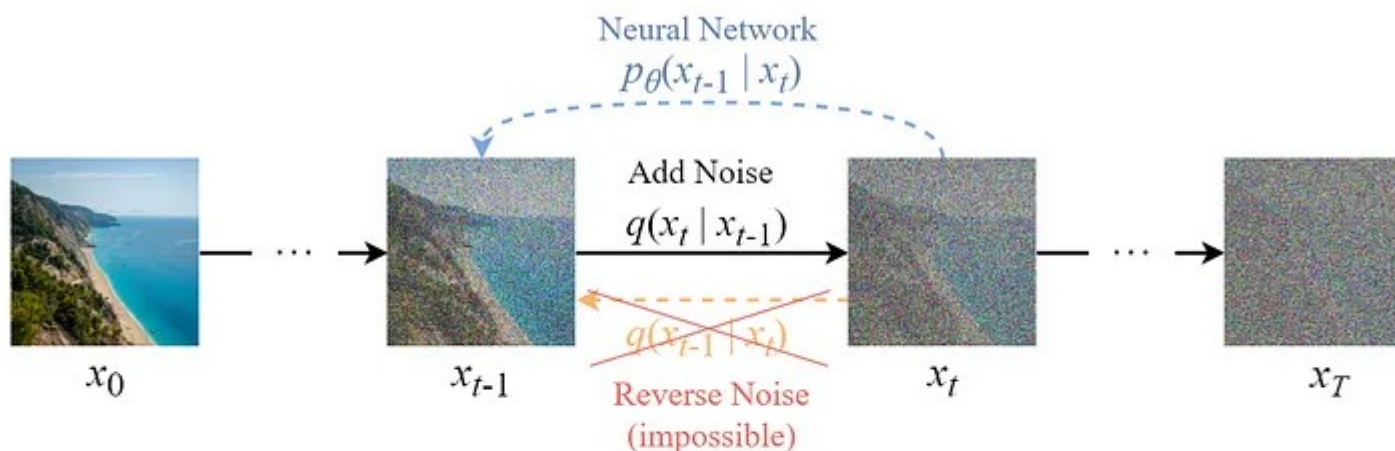
Linear schedule: last steps too noisy and they are not useful

Reverse diffusion process

Reverting one step of the noising process is impossible: we would need all the steps of the noising process

We want a model that allows us to undo the noising process in steps:

- ▶ We train a model to estimate the noise in the sample at instant t
- ▶ From x_0 we can obtain x_{t-1} (using the reparameterization trick)



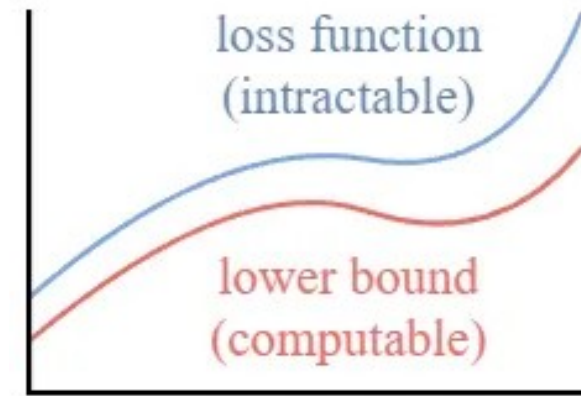
Target Distribution	$q(x_{t-1} x_t) = \mathcal{N}(x_{t-1}; \tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t I)$
Approximated Distribution	$p_\theta(x_{t-1} x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t))$
	$\uparrow$ Learnable parameters (Neural Network)

Reverse diffusion is similar to the decoder in an autoencoder, but the forward process is unparameterized (just adding noise).

The loss function would require having $x_1, x_2, \dots, x_T$, then it is intractable

$$\text{Loss} = -\log(p_\theta(x_0))$$

Depends on $x_1, x_2, \dots, x_T$
Therefore it is intractable!



$$x_t = \sqrt{\bar{\alpha}_t} x_0 + (1 - \bar{\alpha}_t) \epsilon$$

We try to estimate the noise there is in the sample at instant t

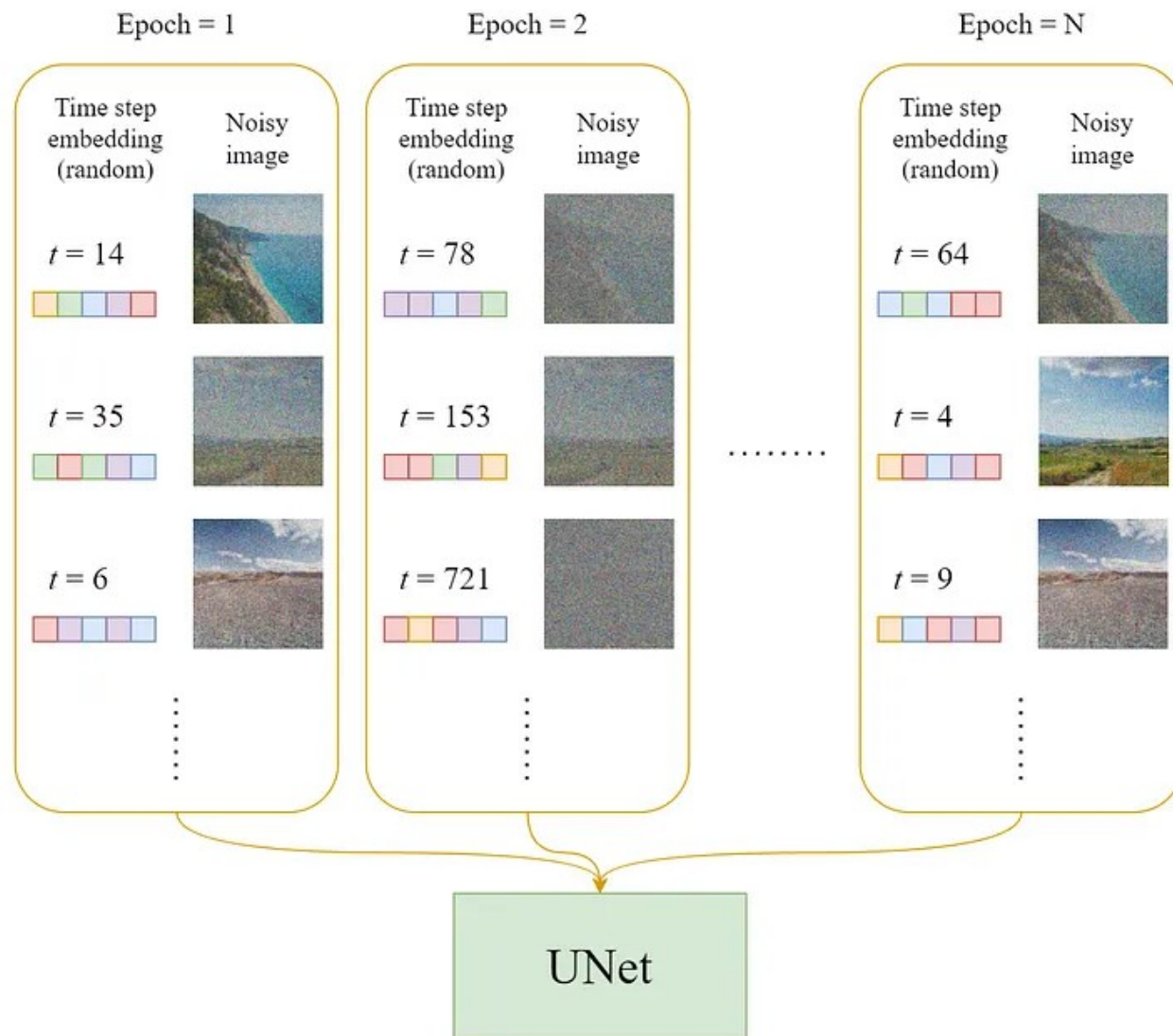
Algorithm 1 Training

- 1: **repeat**
 - 2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
 - 3: $t \sim \text{Uniform}(\{1, \dots, T\})$
 - 4: $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
 - 5: Take gradient descent step on
 $\nabla_\theta \|\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t)\|^2$
 - 6: **until** converged
-

Denoiser training

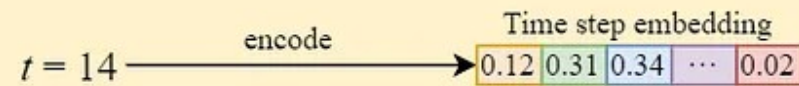
Each epoch:

- ▶ A random time step t is selected for each image
- ▶ The noisy image (corresponding to step t) is generated
- ▶ The variance of the noise corresponding to step t is embedded

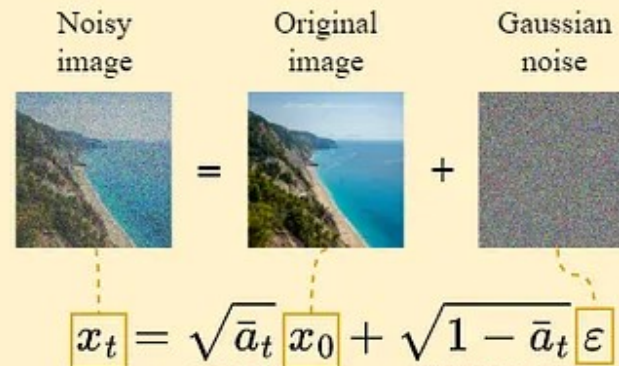


For each training step:

1. Randomly select a time step & encode it



2. Add noise to image



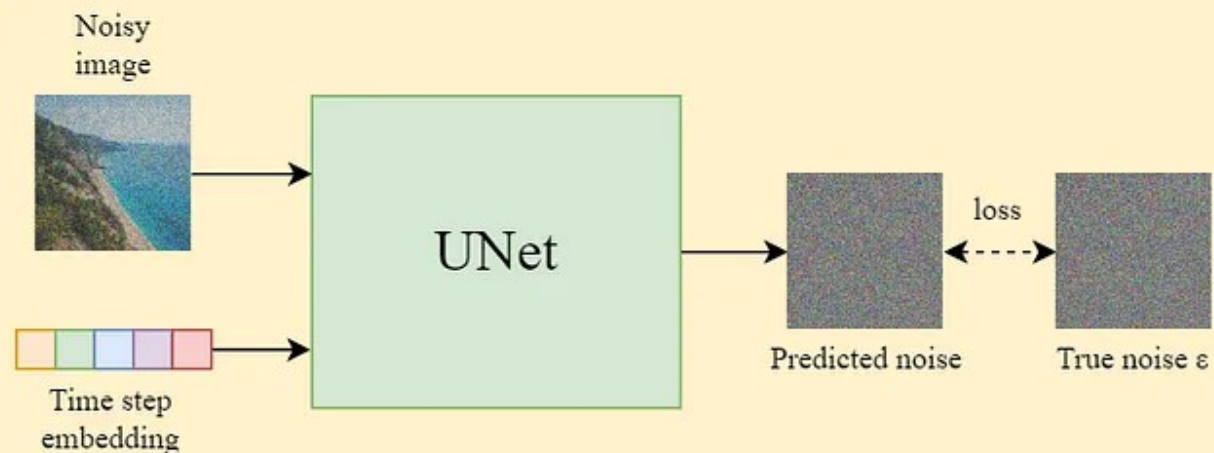
Adjust the amount of noise according to the time step t

$$\epsilon \sim \mathcal{N}(0, 1)$$

$$\alpha_t = 1 - \beta_t$$

$$\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$$

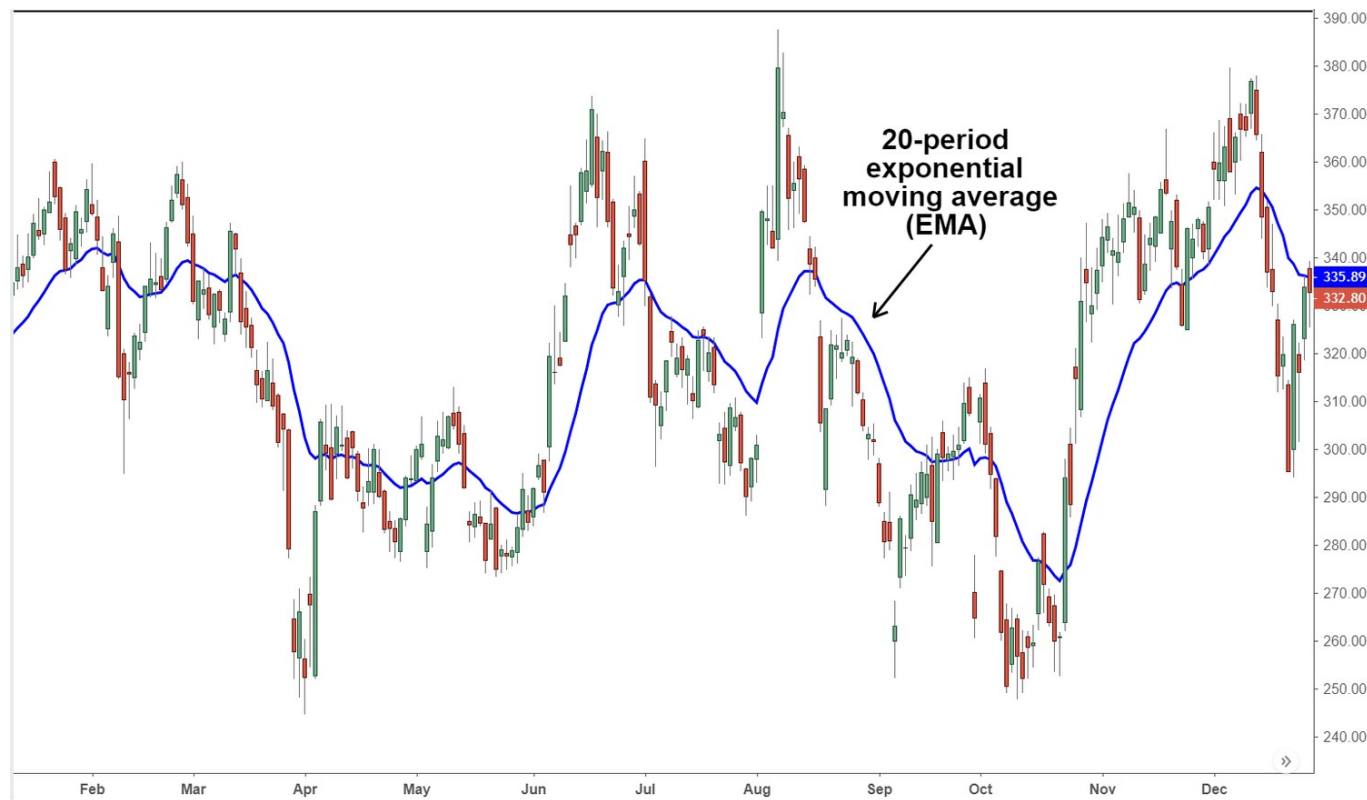
3. Train the UNet



Actually, there are two copies of the network:

- ▶ One is actively trained used gradient descent
- ▶ Another (the EMA network): an exponential moving average (EMA) of the weights of the actively trained network over previous training steps.

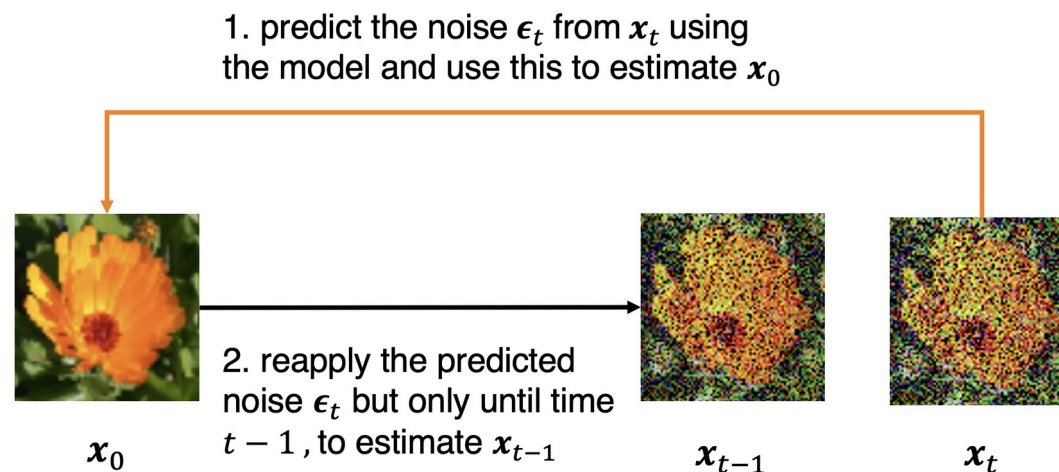
The EMA network is more stable, therefore is used to produce generated output from the network



Sampling

Going from x_t to x_{t-1} is done in two steps:

- ▶ Use the noise prediction to recover an estimate of x_0
- ▶ From this estimate, reapply this noise but only until x_{t-1}



$$\mathbf{x}_{t-1} = \underbrace{\sqrt{\bar{\alpha}_{t-1}} \left(\frac{\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_{\theta}^{(t)}(\mathbf{x}_t)}{\sqrt{\bar{\alpha}_t}} \right)}_{\text{predicted } \mathbf{x}_0} + \underbrace{\sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \cdot \epsilon_{\theta}^{(t)}(\mathbf{x}_t)}_{\text{direction pointing to } \mathbf{x}_t} + \underbrace{\sigma_t \epsilon_t}_{\text{random noise}}$$

Random noise is added in each step. The factor σ_t determines how random the generation process is going to be.

If $\sigma_t = 0$ the generation process will be deterministic (a random noise regenerates always the same image). These are called **denoising diffusion implicit models** (DDIMs)

From a Gaussian noise $N(0, I)$, we iteratively denoise the image

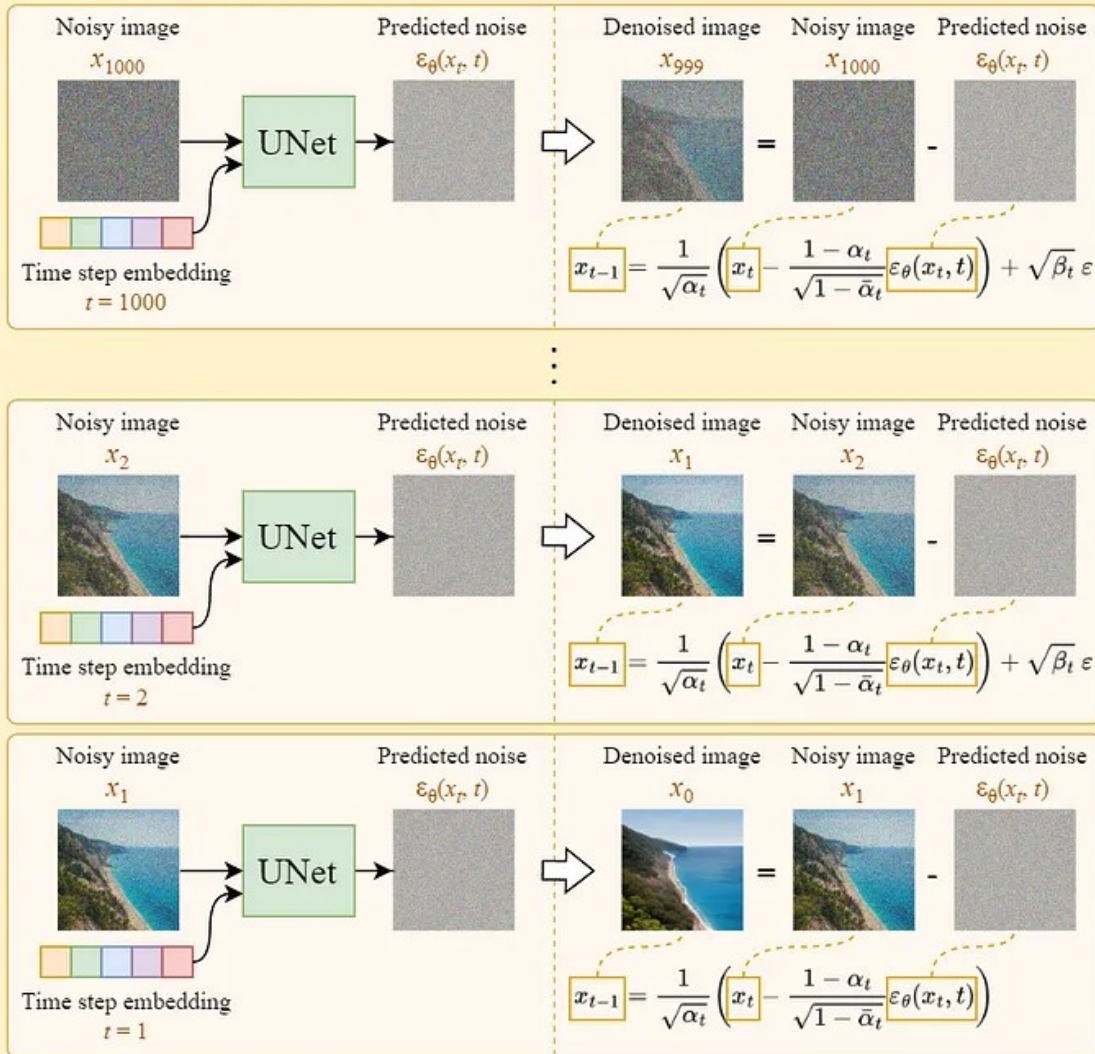
1. Sample a Gaussian noise

$$x_T \sim N(0, I)$$

E.g. $T = 1000$
 $x_{1000} \sim N(0, I)$



2. Iteratively denoise the image



3. Output the denoised image

Denoised image x_0

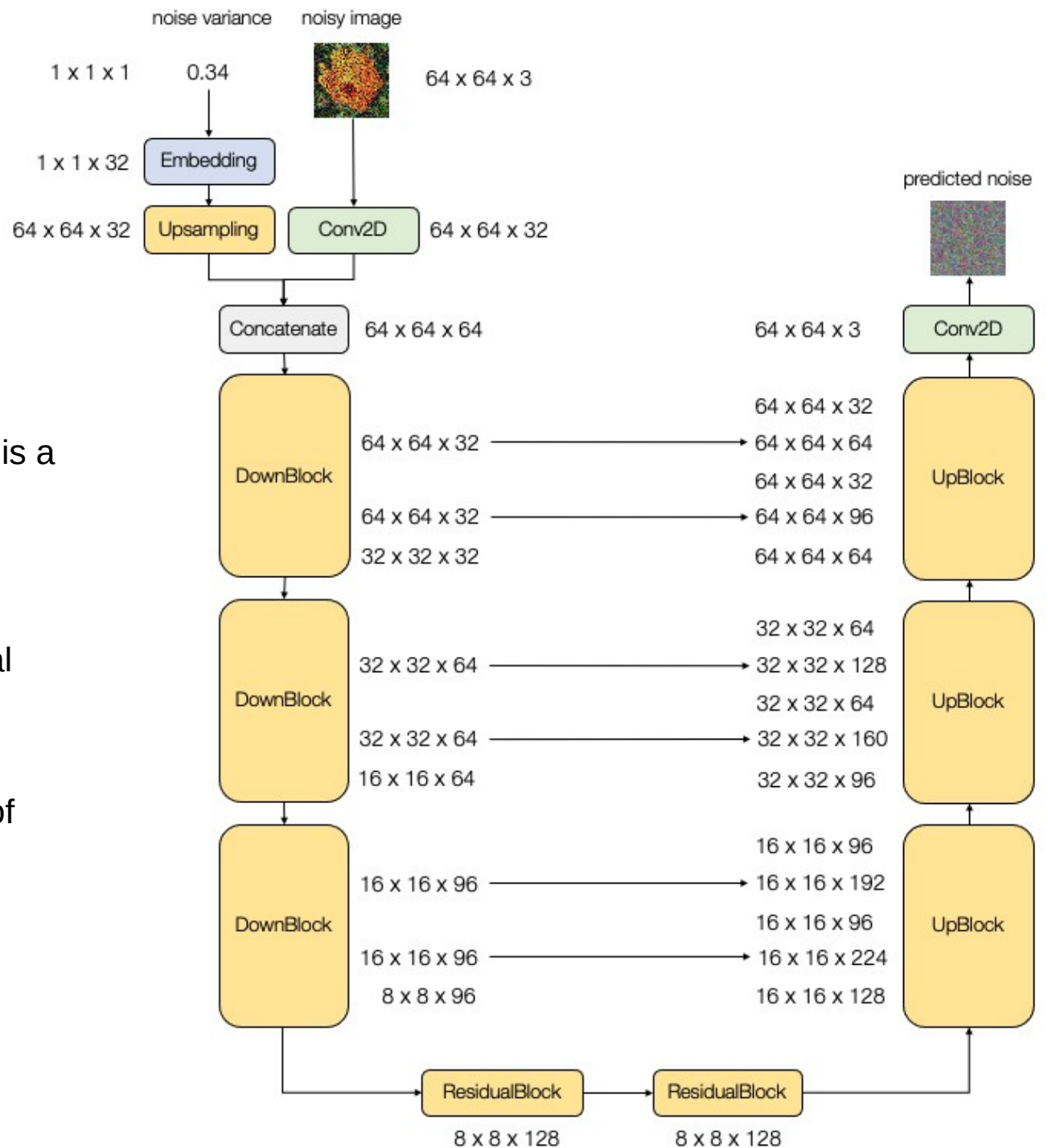


Denoising model

The denoising architecture is a **U-Net** network

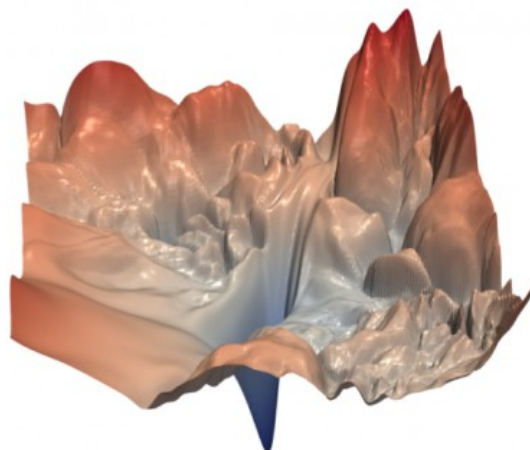
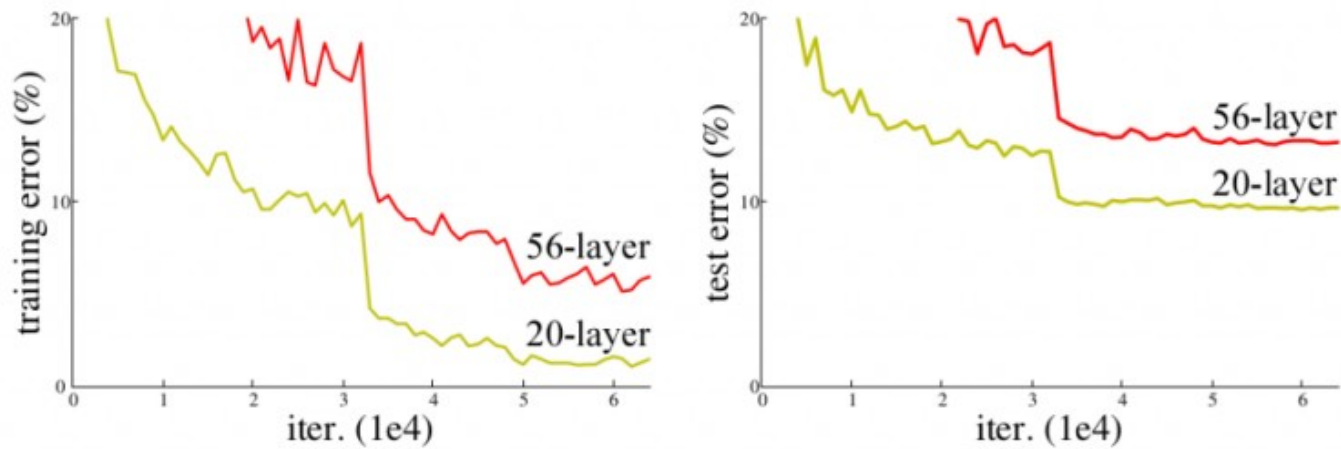
Similar to a VAE (variational autoencoder) with:

- ▶ Skip connections
- ▶ Sinusoidal embedding of the variance
- ▶ Residual blocks
- ▶ Down and upblocks

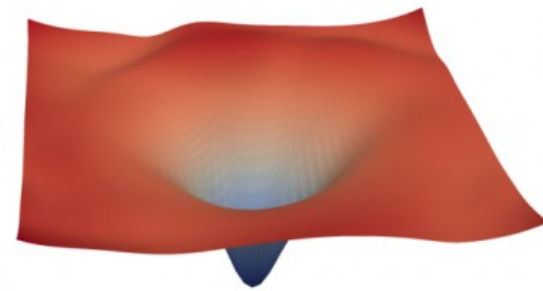


Skip connections reduce *degradation problems*: performance of the model drops down with increase in depth of the architecture

- ▶ Overfitting
- ▶ Vanishing gradient and/or exploding gradient problems



(a) without skip connections



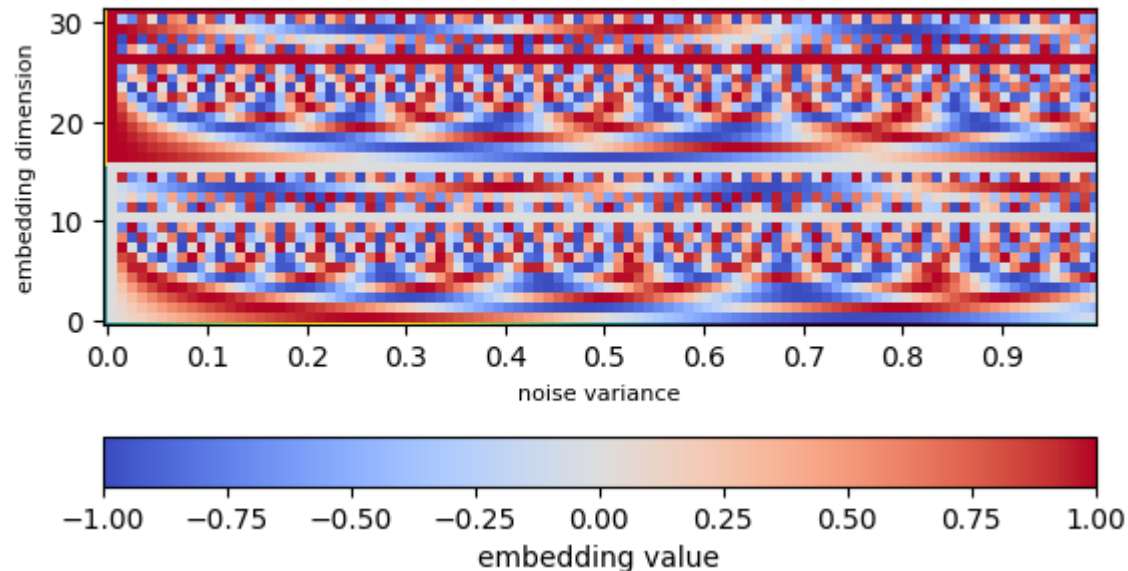
(b) with skip connections

Sinusoidal embedding: converts a scalar value (noise variance) into a higher dimensional vector.

$$\gamma(x) = \left(\sin(2\pi e^{0f}x), \dots, \sin(2\pi e^{(L-1)f}x), \cos(2\pi e^{0f}x), \dots, \cos(2\pi e^{(L-1)f}x) \right)$$

L is half the desired size

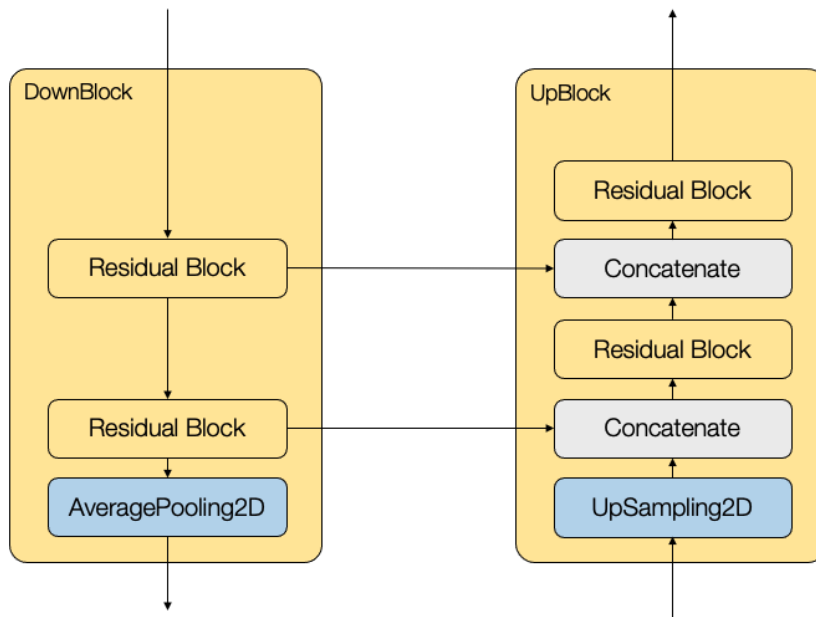
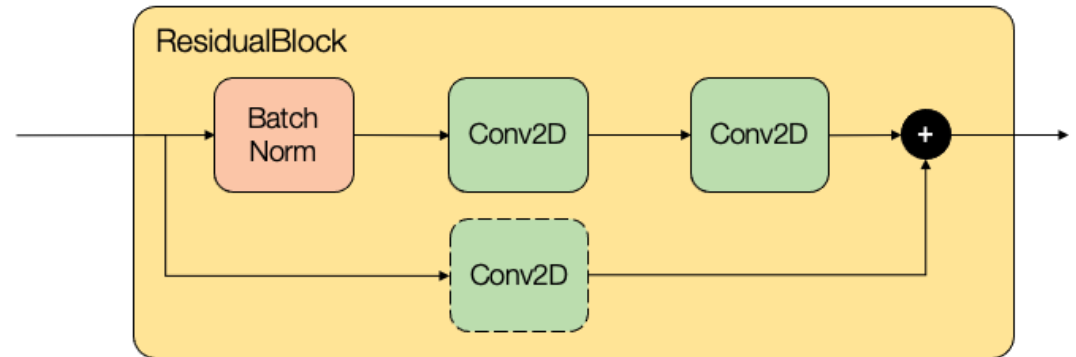
$$f = \frac{\ln(1000)}{L-1} \text{ is a scaling factor}$$



This encodes noise variance as a continuous function and can be very efficiently computed

Residual blocks: groups of layers that contain a skip connection that is combined with the output of the block

They contribute to reduce problems of vanishing or exploding gradients and degradation



Downblocks and upblocks:

- ▶ Downblocks increase the number of channels and reduce the dimension of the images
- ▶ Upblocks do the opposite

Outputs of downblocks are concatenated in the upblocks

5D.3 Examples of diffusion models

Example 1: fashion MNIST

```
(x_train, y_train), _ = fashion_mnist.load_data()

# choose ONE class
TARGET_CLASS = 0 # 0 = t-shirt
mask = (y_train == TARGET_CLASS)
x_train = x_train[mask]

x_train = x_train.astype("float32") / 255.0
x_train = x_train * 2.0 - 1.0
x_train = x_train[..., None]

print("Number of training samples:", x_train.shape[0])

idx = np.random.choice(len(x_train), 16, replace=False)
samples = x_train[idx]

# Convert from [-1, 1] -> [0, 1]
samples = (samples + 1.0) / 2.0
samples = tf.clip_by_value(samples, 0.0, 1.0)

plt.figure(figsize=(4, 4))
for i in range(16):
    img = samples[i, :, :, 0]
    plt.subplot(4, 4, i + 1)
    plt.imshow(1.0 - img, cmap="gray", vmin=0, vmax=1)
    plt.axis("off")

plt.suptitle("Training samples (before diffusion)", fontsize=14)
plt.tight_layout()
plt.show()

dataset = tf.data.Dataset.from_tensor_slices(x_train)
dataset = dataset.shuffle(2048).batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)
```

Training samples (before diffusion)



```
T_STEPS = 150
NUM_EPOCHS = 200
BATCH_SIZE = 64
LEARNING_RATE = 5e-5
EMA_DECAY = 0.999
```

Number of training samples: 6000

Diffusion schedule: linear $\beta_1=0.0001 \dots \beta_T=0.02$

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon$$

$$\alpha_t = 1 - \beta_t \quad \bar{\alpha}_t = \prod_{i=1}^t \alpha_i$$

```
betas = np.linspace(1e-4, 0.02, T_STEPS, dtype=np.float32)
alphas = 1.0 - betas
alpha_bar = np.cumprod(alphas)

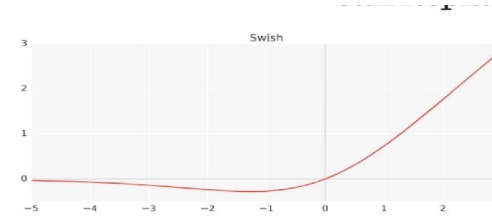
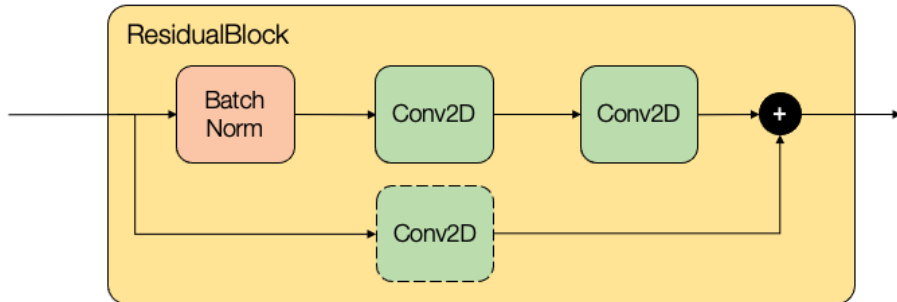
sqrt_alpha_bar = np.sqrt(alpha_bar)
sqrt_one_minus_alpha_bar = np.sqrt(1 - alpha_bar)
```

Sinusoidal embedding: $\gamma(x) = \left(\sin(2\pi e^{0f}x), \dots, \sin(2\pi e^{(L-1)f}x), \cos(2\pi e^{0f}x), \dots, \cos(2\pi e^{(L-1)f}x) \right)$

```
class TimeEmbedding(layers.Layer):
    def __init__(self, dim=128):
        super().__init__()
        self.dim = dim

    def call(self, t):
        half = self.dim // 2
        freqs = tf.exp(
            -tf.math.log(10000.0) * tf.range(half, dtype=tf.float32) / half
        )
        t = tf.cast(t[:, None], tf.float32)
        args = t * freqs[None, :]
        return tf.concat([tf.sin(args), tf.cos(args)], axis=-1)
```

Residual blocks:



Swish Activation Function

$$f(x) = x * \text{sigmoid}(x)$$

```
class ResBlock(layers.Layer):
    def __init__(self, out_channels):
        super().__init__()
        self.out_channels = out_channels
        self.norm1 = layers.GroupNormalization(8)
        self.conv1 = layers.Conv2D(out_channels, 3, padding="same")
        self.norm2 = layers.GroupNormalization(8)
        self.conv2 = layers.Conv2D(out_channels, 3, padding="same")
        self.skip = None

    def build(self, input_shape):
        in_channels = input_shape[-1]
        if in_channels != self.out_channels:
            self.skip = layers.Conv2D(self.out_channels, 1, padding="same")
        else:
            self.skip = lambda x: x

    def call(self, x):
        h = tf.nn.swish(self.norm1(x))
        h = self.conv1(h)
        h = tf.nn.swish(self.norm2(h))
        h = self.conv2(h)
        return self.skip(x) + h
```


U-Net denoising model:

```
def build_unet():
    x_in = layers.Input((28, 28, 1))
    t_in = layers.Input(), dtype=tf.int32

    # ----- Time embedding -----
    t_emb = TimeEmbedding(128)(t_in)
    t_emb = layers.Dense(256, activation="swish")(t_emb)

    t_emb_256 = layers.Dense(256, activation="swish")(t_emb)
    t_emb_256 = layers.Reshape((1, 1, 256))(t_emb_256)

    t_emb_384 = layers.Dense(384, activation="swish")(t_emb)
    t_emb_384 = layers.Reshape((1, 1, 384))(t_emb_384)

    # ----- Encoder -----
    x = layers.Conv2D(64, 3, padding="same")(x_in)
    r1 = ResBlock(64)(x) # 28x28, 64 ch

    x = layers.AveragePooling2D(2)(r1) # 14x14
    r2 = ResBlock(128)(x) # 128 ch

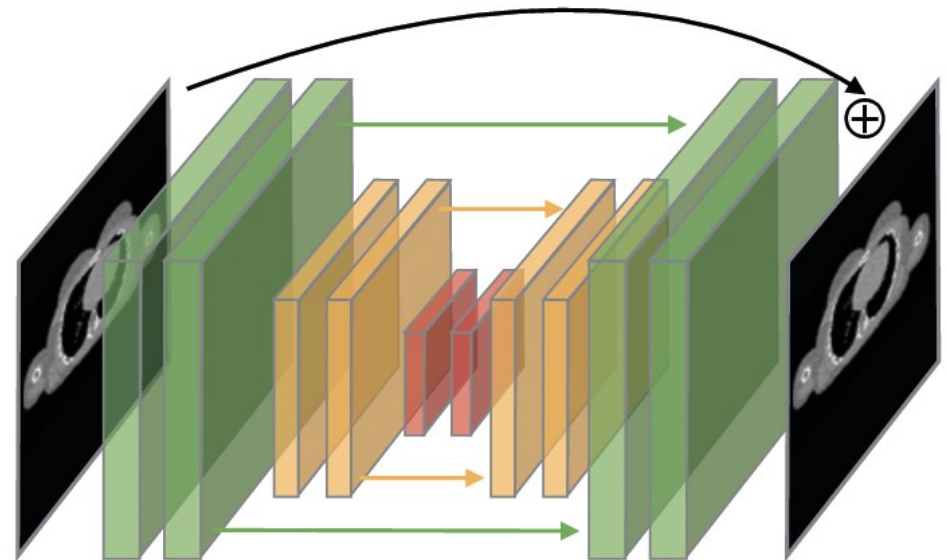
    x = layers.AveragePooling2D(2)(r2) # 7x7
    x = ResBlock(256)(x) # 256 ch
    x = x + t_emb_256

    # ----- Decoder -----
    x = layers.UpSampling2D()(x) # 14x14
    x = layers.Concatenate()([x, r2]) # 384 ch
    x = x + t_emb_384
    x = ResBlock(128)(x)

    x = layers.UpSampling2D()(x) # 28x28
    x = layers.Concatenate()([x, r1]) # 192 ch
    x = ResBlock(64)(x)

    out = layers.Conv2D(1, 3, padding="same")(x)
    return keras.Model([x_in, t_in], out)

model = build_unet()
```



Total params: 2,222,849 (8.48 MB)

EMA (Exponential Moving Average):

```
ema_model = build_unet()
ema_model.set_weights(model.get_weights())

def update_ema_weights(model, ema_model, decay):
    for w, ema_w in zip(model.weights, ema_model.weights):
        ema_w.assign(decay * ema_w + (1.0 - decay) * w)
```

Forward diffusion:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$$

```
def q_sample(x0, t, noise):
    return (
        extract(sqrt_alpha_bar, t) * x0 +
        extract(sqrt_one_minus_alpha_bar, t) * noise
    )
```

Training:

```
optimizer = keras.optimizers.Adam(LEARNING_RATE)

@tf.function
def train_step(x0):
    t = tf.random.uniform((tf.shape(x0)[0],), 0, T_STEPS, tf.int32)
    noise = tf.random.normal(tf.shape(x0))
    x_t = q_sample(x0, t, noise)

    with tf.GradientTape() as tape:
        pred_noise = model([x_t, t], training=True)
        loss = tf.reduce_mean((noise - pred_noise) ** 2)

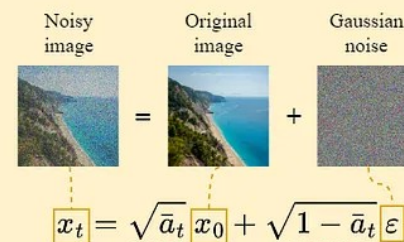
    grads = tape.gradient(loss, model.trainable_variables)
    optimizer.apply_gradients(zip(grads, model.trainable_variables))
    return loss
```

```
for epoch in range(NUM_EPOCHS):
    for batch in dataset:
        loss = train_step(batch)
        update_ema_weights(model, ema_model, EMA_DECAY)
```

1. Randomly select a time step & encode it



2. Add noise to image



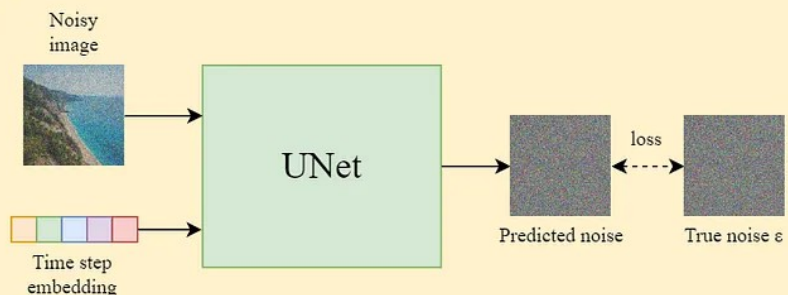
Adjust the amount of noise according to the time step t

$$\epsilon \sim \mathcal{N}(0, 1)$$

$$\alpha_t = 1 - \beta_t$$

$$\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$$

3. Train the UNet



Generate samples:

```
def sample(model, n=16):
    x = tf.random.normal((n, 28, 28, 1))
    for t in reversed(range(T_STEPS)):
        t_batch = tf.fill((n,), t)
        pred = model([x, t_batch], training=False)

        alpha = alphas[t]
        alpha_bar_t = alpha_bar[t]

        x = (1 / tf.sqrt(alpha)) * (
            x - (1 - alpha) / tf.sqrt(1 - alpha_bar_t) * pred
        )

        if t > 0:
            x += tf.sqrt(betas[t]) * tf.random.normal(tf.shape(x))

    return x
```

Generated samples

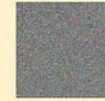


1. Sample a Gaussian noise

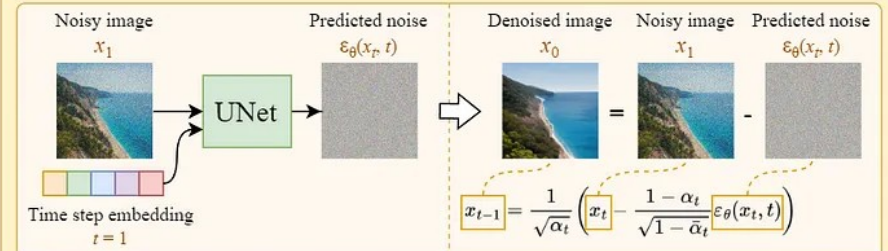
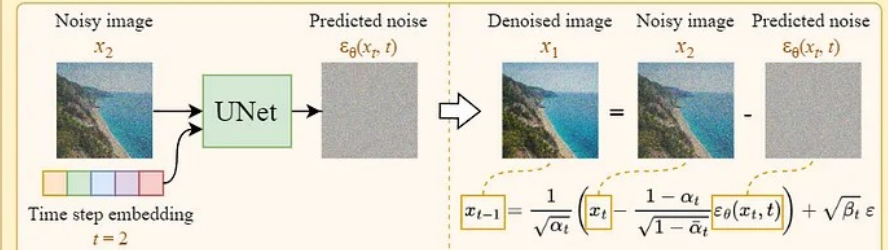
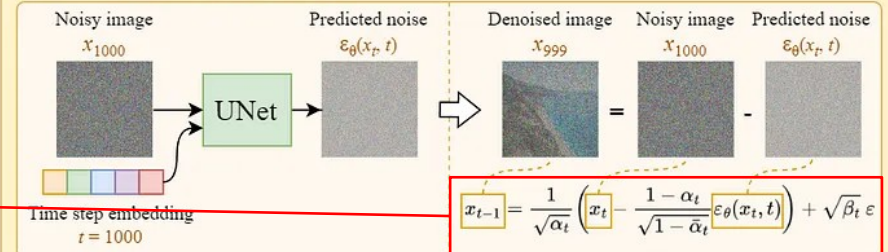
$$x_T \sim N(0, I)$$

E.g. $T = 1000$

$$x_{1000} \sim N(0, I)$$



2. Iteratively denoise the image

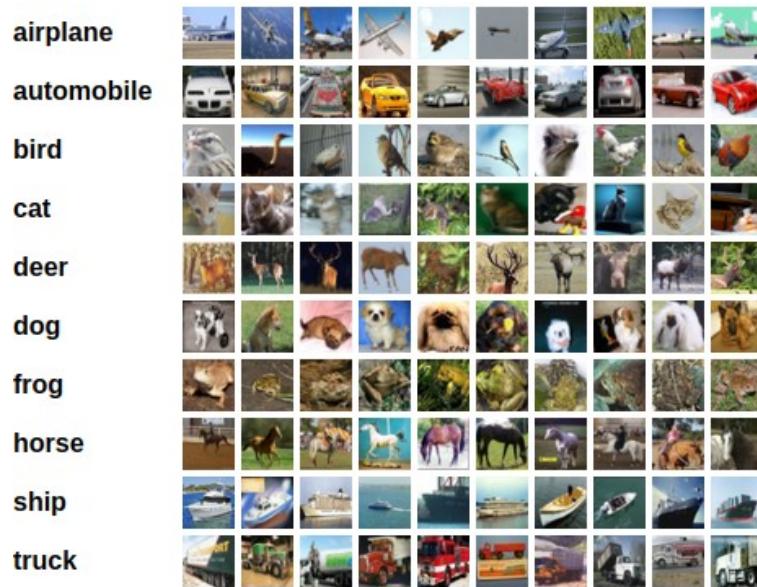


3. Output the denoised image

Denoised image x_0



Example 2: images of cars from CIFAR-10



```
(X_train, y_train), (X_test, y_test) = tf.keras.datasets.cifar10.load_data()  
X_train = X_train[y_train.squeeze() == 1]  
X_train = (X_train / 127.5) - 1.0
```

Selects only
automobiles

```
IMG_SIZE = 32      # input image size, CIFAR-10 is 32x32  
BATCH_SIZE = 128   # for training batch size  
timesteps = 16     # how many steps for a noisy image into clear  
time_bar = 1 - np.linspace(0, 1.0, timesteps + 1) # linspace for timesteps
```

```
def forward_noise(x, t):  
    a = time_bar[t]      # base on t  
    b = time_bar[t + 1]  # image for t + 1  
  
    noise = np.random.normal(size=x.shape) # noise mask  
    a = a.reshape((-1, 1, 1, 1))  
    b = b.reshape((-1, 1, 1, 1))  
    img_a = x * (1 - a) + noise * a  
    img_b = x * (1 - b) + noise * b  
    return img_a, img_b  
  
def generate_ts(num):  
    return np.random.randint(0, timesteps, size=num)
```

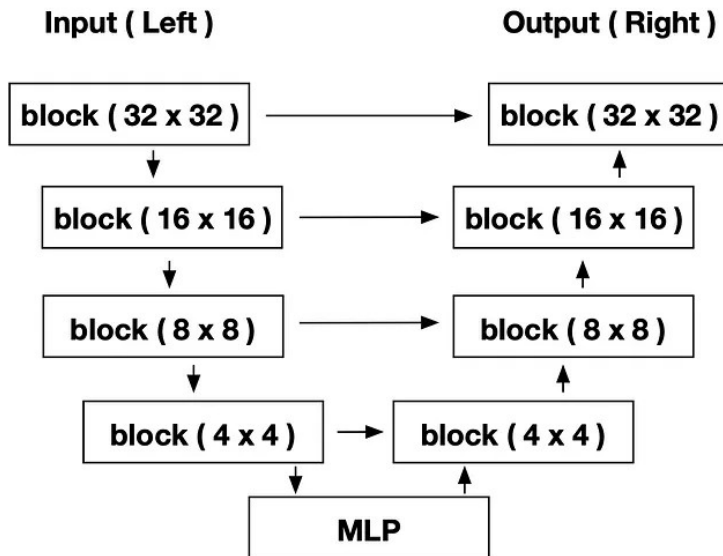
x is a batch of clean images
t is a batch of random instants

img_a: images x in instants t
img_b: images x in instants t+1

We have a linear noise schedule

<https://tree.rocks/make-diffusion-model-from-scratch-easy-way-to-implement-quick-diffusion-model-e60d18fd0f2e>

The time instants are always an input to the block



```
def block(x_img, x_ts):
    x_parameter = layers.Conv2D(128, kernel_size=3, padding='same')(x_img)
    x_parameter = layers.Activation('relu')(x_parameter)

    time_parameter = layers.Dense(128)(x_ts)
    time_parameter = layers.Activation('relu')(time_parameter)
    time_parameter = layers.Reshape((1, 1, 128))(time_parameter)
    x_parameter = x_parameter * time_parameter

    # -----
    x_out = layers.Conv2D(128, kernel_size=3, padding='same')(x_img)
    x_out = x_out + x_parameter
    x_out = layers.LayerNormalization()(x_out)
    x_out = layers.Activation('relu')(x_out)

    return x_out
```

Trains one batch
NOTE: the model tries to obtain images

```
def train_one(x_img):
    x_ts = generate_ts(len(x_img))
    x_a, x_b = forward_noise(x_img, x_ts)
    loss = model.train_on_batch([x_a, x_ts], x_b)
    return loss
```

```
def train(R=50):
    bar = trange(R)
    total = 100
    for i in bar:
        for j in range(total):
            x_img = X_train[np.random.randint(len(X_train), size=BATCH_SIZE)]
            loss = train_one(x_img)
            pg = (j / total) * 100
            if j % 5 == 0:
                bar.set_description(f'loss: {loss:.5f}, p: {pg:.2f}%')
```

Trains one epoch

Training

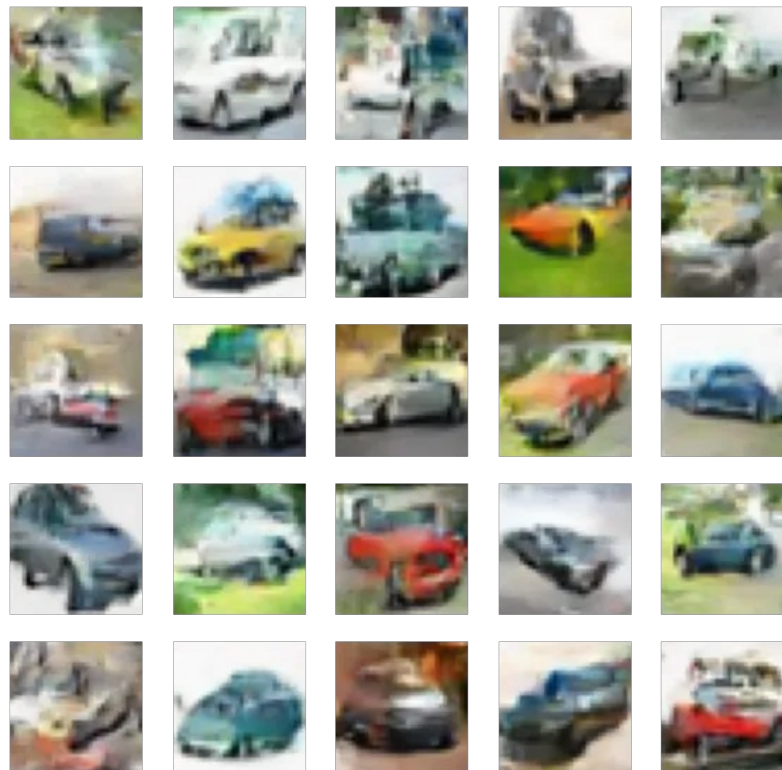
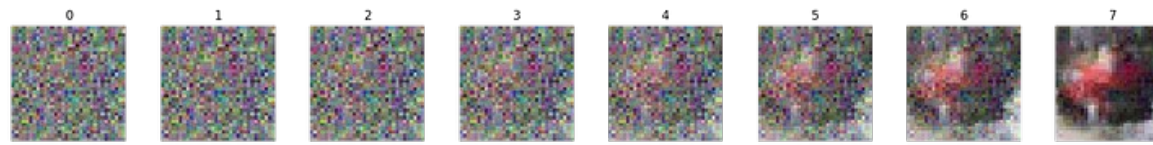
```
for _ in range(10):
    train()
    # reduce learning rate for next training
    model.optimizer.learning_rate = max(0.000001, model.optimizer.learning_rate * 0.9)
```

```
def predict(x_idx=None):
    x = np.random.normal(size=(32, IMG_SIZE, IMG_SIZE, 3))

    for i in trange(timesteps):
        t = i
        x = model.predict([x, np.full((32), t)], verbose=0)
    show_examples(x)
```

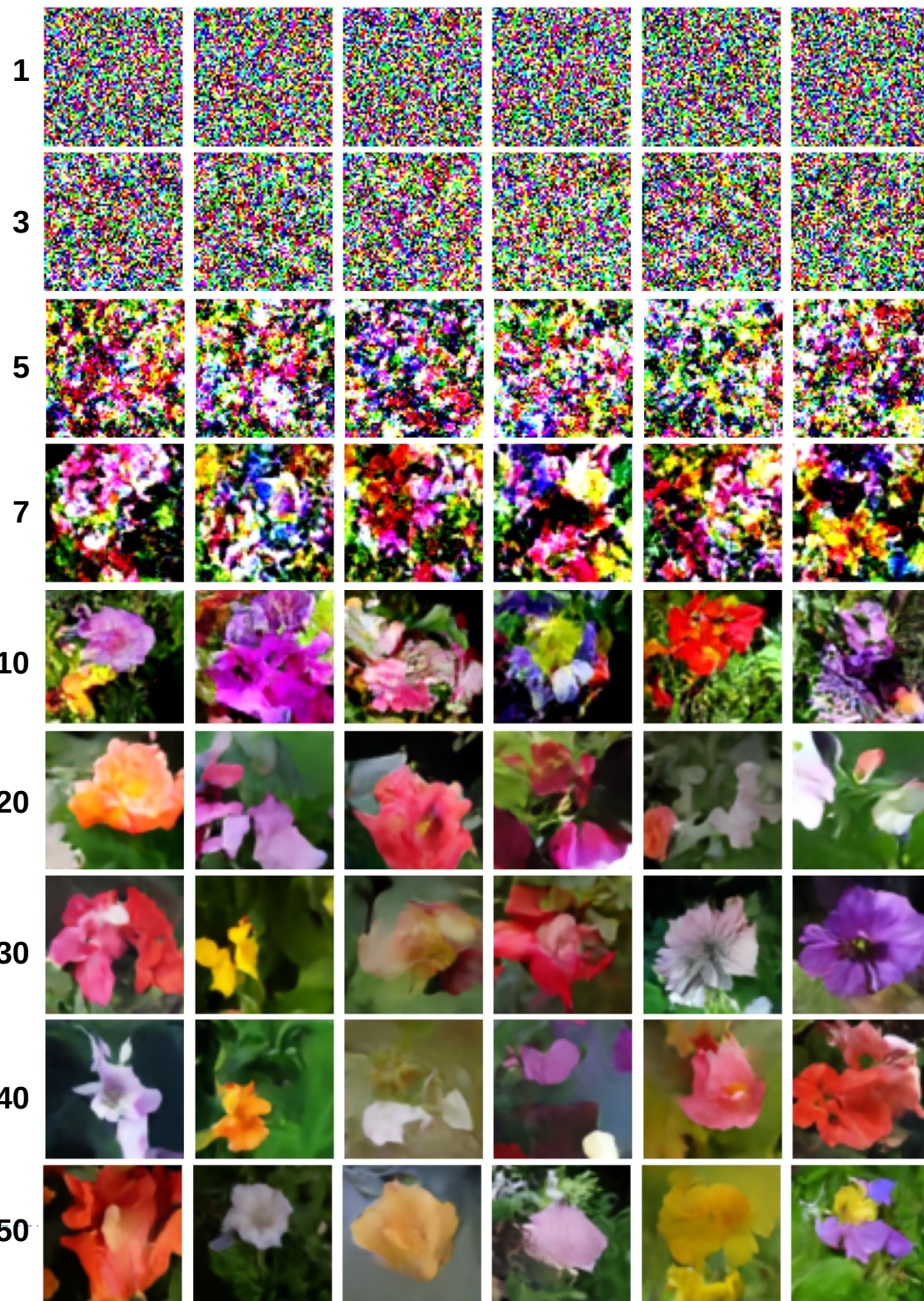
Batch of noise

Iterative denoising



Example 3: Oxford 102 flower dataset

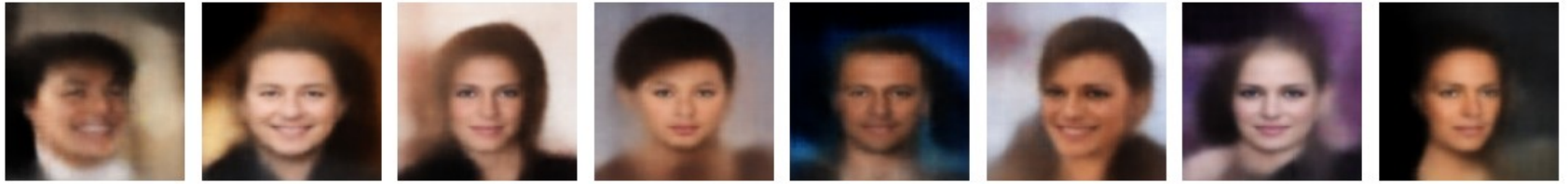
Oxford 102 flower dataset: 102 classes of flowers from the UK, over 8000 images



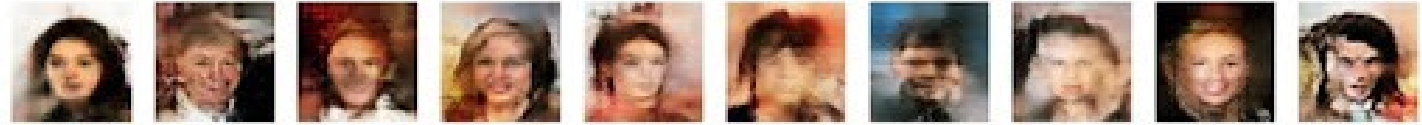
Example 4: CelebA dataset

CelebA dataset: over 200000 photographs of celebrities

VAEs



GANs



Diffusion models



	VAEs	GANs	Diffusion models
Pros	Fast generation rate Diverse sample generation	Fast generation rate High generation quality	High generation quality Diverse sample generation
Cons	Low generation quality	Unstable training Low generation diversity (mode collapse)	Not so fast generation rate

5D.4 Stable Diffusion

Multimodal learning: generative models to convert between different kinds of data

MIDJOURNEY



DALL-E 2

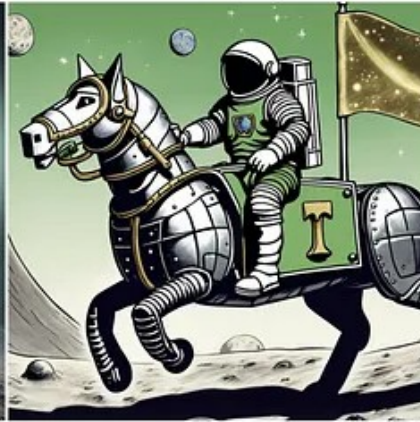


STABLEDIFFUSION



Hubble Telescope photograph of an incredible nebula, deep space photography, astonishing photo, wormholes and nebulas

<https://hackernoon.com/dall-e-2-vs-midjourney-vs-stable-diffusion-battle-of-the-gods>

DALL-E 3**MIDJOURNEY 5.2****STABLE XL**

Prompt: An astronaut riding a steel horse on the moon. The astronaut is wearing a medieval armor with a party hat and a green sword.

DALL-E 3**MIDJOURNEY 5.2****STABLE XL**

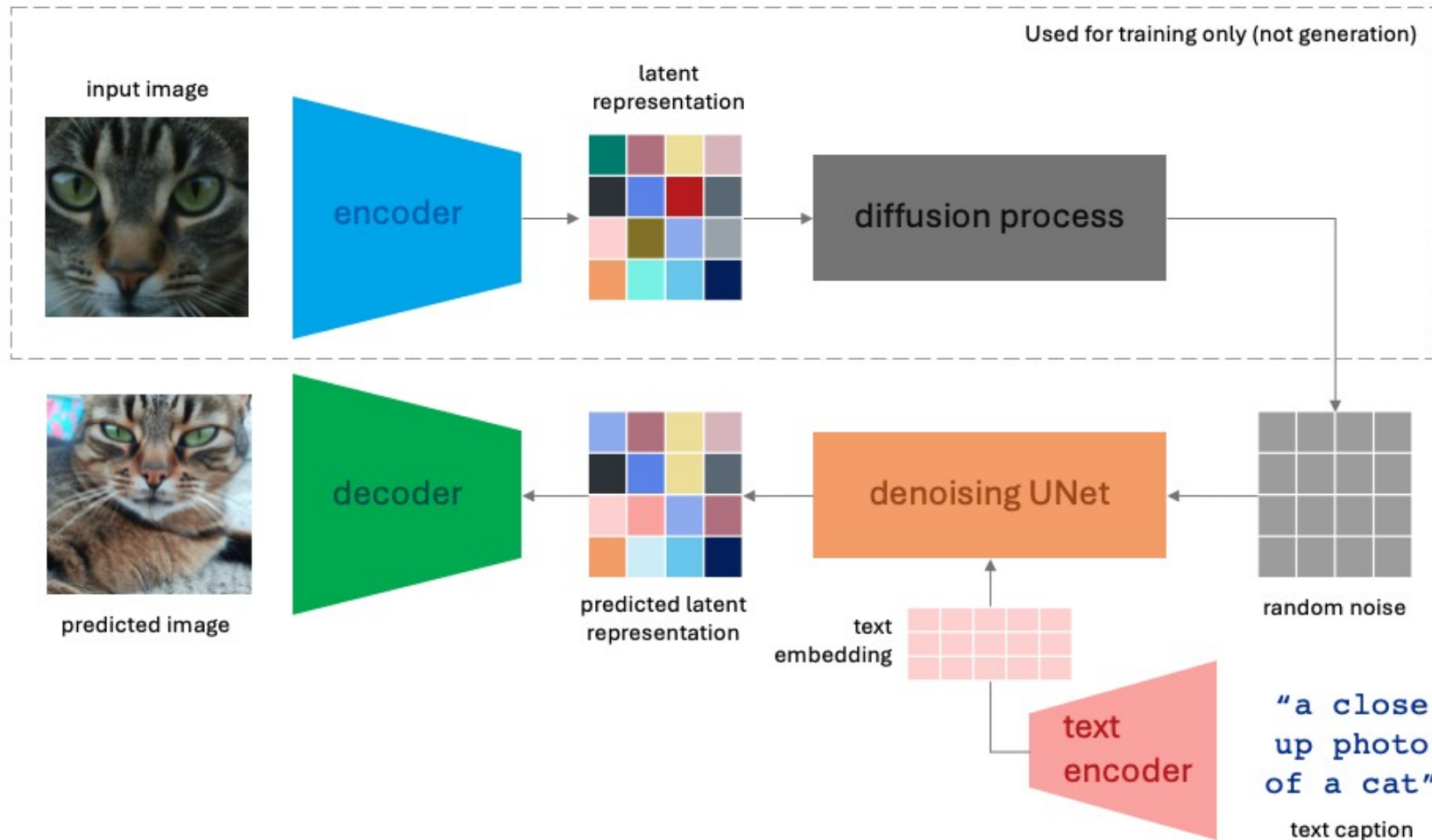
Prompt: A dreamy landscape where clouds are made of cotton candy and rivers flow with liquid gold

<https://generativeai.pub/dall-e-3-vs-midjourney-5-2-vs-stable-xl-same-prompt-different-results-a68ae19b223e>

The original name of Stable Diffusion is “**Latent Diffusion Model**” (LDM)

The diffusion process happens in latent space: it combines a diffusion model and an autoencoder

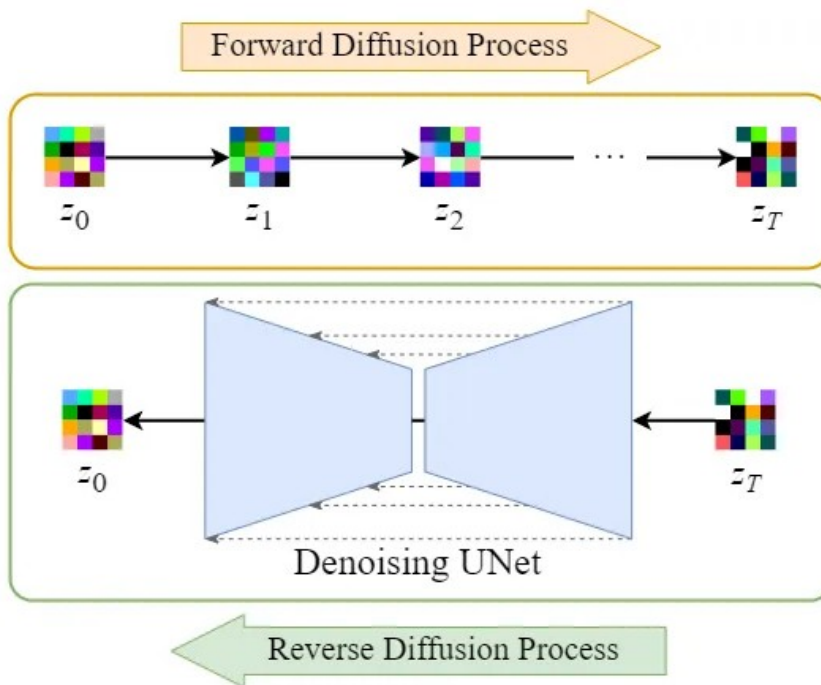
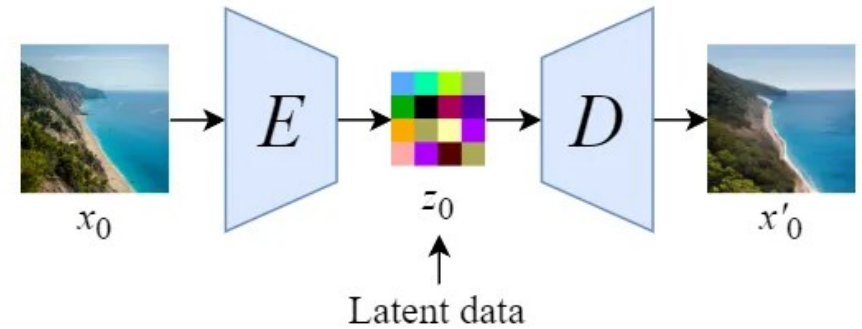
Code and model weights available publicly through HuggingFaces <https://huggingface.co/stabilityai/stable-diffusion-2-1>



Latent space

An autoencoder compresses the image data into lower-dimensional representations:

- ▶ By using the trained encoder E , we can encode the full-sized image into lower dimensional latent data (compressed data).
- ▶ By using the trained decoder D , we can decode the latent data back into an image



The forward and reverse diffusion processes will be done in the latent space.

This keeps the model lightweight

Training data

Stable Diffusion was trained on pairs of images and captions taken from LAION-5B, a publicly available dataset derived from [Common Crawl](#) data scraped from the web, where **5 billion image-text pairs** were classified based on language and filtered into separate datasets by resolution, a predicted likelihood of containing a watermark, and predicted "aesthetic" score (e.g. subjective visual quality).^[13] The dataset was created by [LAION](#), a German non-profit which receives funding from Stability AI.^{[13][14]} The Stable Diffusion model was trained on three subsets of LAION-5B: laion2B-en, laion-high-resolution, and laion-aesthetics v2 5+.^[13] A third-party analysis of the model's training data identified that out of a smaller subset of 12 million images taken from the original wider dataset used, approximately 47% of the sample size of images came from 100 different domains, with **Pinterest** taking up 8.5% of the subset, followed by websites such as **WordPress, Blogspot, Flickr, DeviantArt** and **Wikimedia Commons**.^{[15][13]}



Original:



Generated:

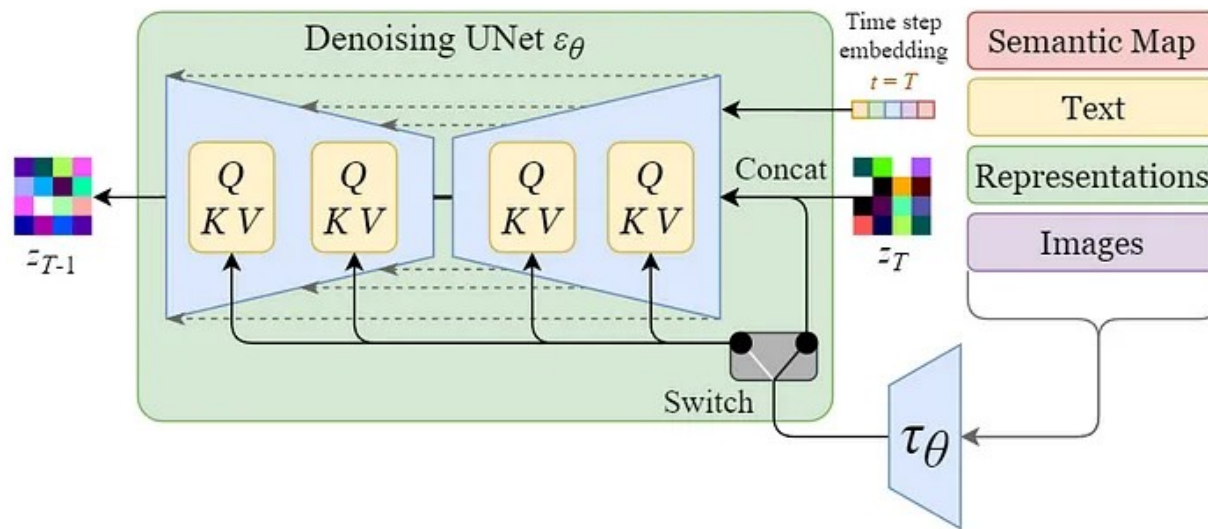
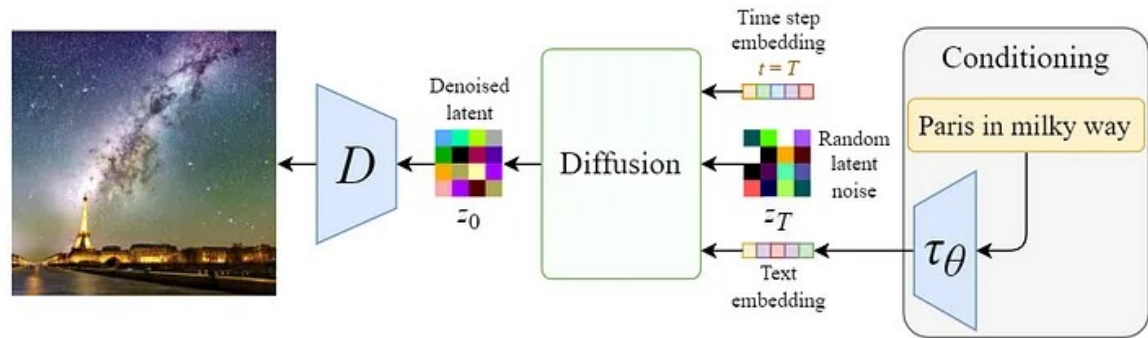


The model was trained using 256 [Nvidia A100](#) GPUs on [Amazon Web Services](#) for a total of 150,000 GPU-hours, at a cost of \$600,000.^{[31][32][33]}



Conditioning

Stable diffusion generates images from text prompts. This is performed by accepting conditioning inputs



Text inputs, they are first converted into embeddings (vectors) using a language model τ_θ (e.g. CLIP), and then they are mapped into the U-Net via the (multi-head) Attention(Q, K, V) layer.

$$\boxed{\nabla_{\theta} \|\epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, t)\|^2} \quad \text{Diffusion model}$$

Training procedure

Similar to a diffusion model with two main changes:

- ▶ Input latent data z_t instead of the image x_t .
- ▶ Added conditioning $\tau\theta(y)$ of the text prompts to the U-Net.

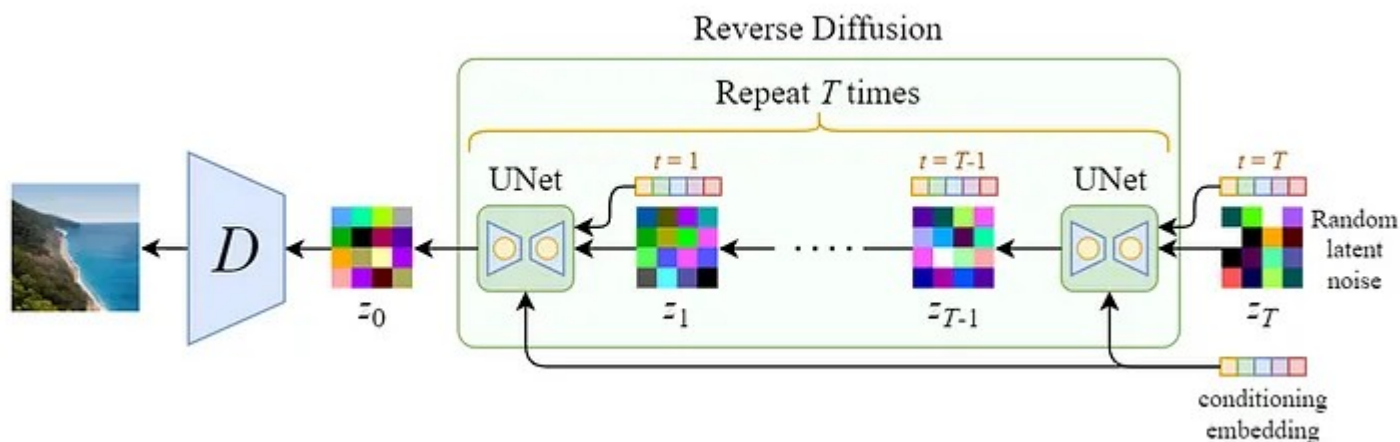
$$z_0 = E(x_0)$$

$$z_t = \sqrt{\bar{\alpha}_t} z_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$$

$$L_{\text{LDM}} = \mathbb{E}_{t, z_0, \epsilon, y} \left[\|\epsilon - \epsilon_{\theta}(z_t, t, \tau\theta(y))\|^2 \right]$$

Conditioning

Sampling





painting of a human cyborg in a city salvador dali highly detailed



clown eating a carrot spooky lighting hyperrealistic



Darth Vader cooking an omelette



cat jumping over a cow highly detailed

<https://stablediffusionweb.com/app/image-generator>

A doctor interpreting an X-ray



Reality is biased... is it a good idea compensate this bias in the models?

From Black Nazis to female Popes and American Indian Vikings: How AI went 'woke'

As Google comes under fire after its Gemini bot exhibited bias, how can new technology avoid these errors?

Matthew Field

23 February 2024 • 8:00pm

